

Mapeamento de Elementos HTML - Google

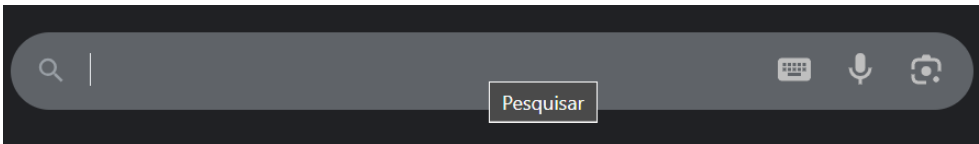
Objetivo

Explorar e mapear elementos da interface do mecanismo de busca **Google**, aplicando boas práticas de seleção para automação de testes. A escolha dos seletores priorizou atributos semânticos e identificadores estáveis, evitando seletores frágeis ou altamente dependentes da estrutura do DOM.

Site: <https://www.google.com/>

Elementos Mapeados

1º Elemento: Campo de Busca

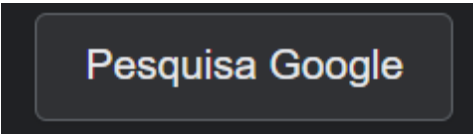


- **Nome do elemento:** Campo principal de texto para busca.
- **HTML Relevante:**

```
<input
id="APjFqb"
name="q"
aria-label="Pesquisar"
class="gLfyf"
/>
```

- **ELEMENT:** `<textarea id="APjFqb" name="q"></textarea>`
- **SELECTOR:** `#APjFqb`
- **XPATH:** `//*[@id='APjFqb']`
- **Estratégia:** ID direto.
- **Alternativas consideradas:**
 - `[aria-label="Pesquisar"]`
 - `[title="Pesquisar"]`
- **Justificativa:** O `id` é a opção mais robusta por ser um identificador direto e único no DOM, ideal para localizar o campo de busca. `name="q"` é uma excelente alternativa usada pelo backend do Google, seguido por `aria-label` e `title`, que ajudam na acessibilidade e podem ser utilizados caso o ID mude. O uso de classes (como `.gLfyf`) é desaconselhado, pois essas classes costumam ser geradas automaticamente e são imprevisíveis.

2º Elemento: Botão "Pesquisa Google"



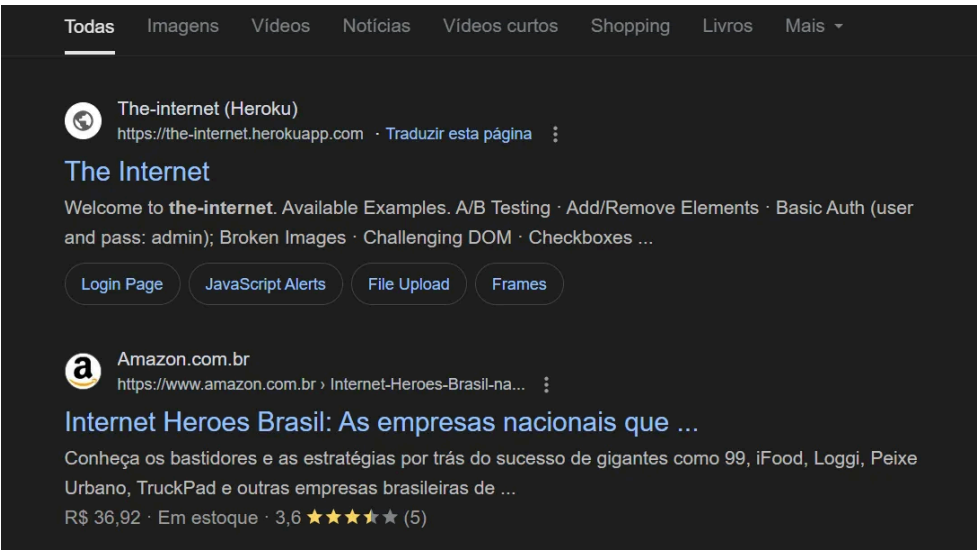
- **Nome do elemento:** Botão de envio da busca principal.
- **HTML Relevante:**

```
<input
name="btnK"
value="Pesquisa Google"
aria-label="Pesquisa Google"
type="submit"
```

```
class="gNO89b"
/>
```

- **ELEMENT:** `<input name="btnK" value="Pesquisa Google">`
- **SELECTOR:** `[name='btnK']`
- **XPATH:** `//input[@name='btnK']`
- **Estratégia:** Atributo `name` .
- **Alternativas consideradas:**
 - `[aria-label="Pesquisa Google"]`
 - `[value="Pesquisa Google"]`
- **Justificativa:** O `name` é um seletor funcional e bem definido, geralmente utilizado internamente para referência ao botão de envio, portanto sendo uma escolha robusta. `aria-label` e `value` reforçam o mapeamento e oferecem opções acessíveis. Classes (ex: `input.gNO89b[type="submit"]`) podem ser frágeis pelo risco de obsolescência.

3º Elemento: Links de Resultados da Busca (Array de Links)



- **Nome do elemento:** Título clicável de resultado da busca.
- **HTML Relevante:**

```
<h3 class="LC20Ib">
  <a href="https://example.com">Título do Resultado</a>
</h3>
```

- **ELEMENT:** `<a href="https://example.com">Título do Resultado</a>`
- **SELECTOR:** `h3.LC20Ib > a`
- **XPATH:** `//h3[contains(@class, 'LC20Ib')]/a`
- **Estratégia:** Hierarquia CSS baseada em padrão semântico.
- **Estratégia alternativa considerada:** `div.g > div.rc > h3 > a` .
- **Justificativa:** O seletor `h3.LC20Ib > a` é conciso e eficaz para capturar os links principais dos resultados. O uso de `div.g` ou `div.rc` como escopo é uma boa prática adicional para iterar blocos de resultados. XPath pode ser usado em casos onde CSS falhe, mas é menos legível.

Considerações Finais

- Apesar de muitos seletores do Google parecerem dinâmicos, há atributos estáveis como `id` , `name` e `aria-label` que devem ser priorizados sempre que possível.
- O uso de esperas dinâmicas é essencial para garantir que o DOM esteja pronto antes da interação, principalmente ao manipular resultados que são renderizados após o carregamento inicial.
- Nenhum dos elementos analisados utiliza `data-testid` , o que exige maior atenção ao contexto e ao comportamento dos seletores escolhidos.

- XPath foi usado apenas quando necessário, priorizando sempre seletores CSS claros e diretos.